FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

MIDTERM

Charles Gabriel Marcelo	Doc. Hadju Tejuco
Name of Student	Name of Professor
October 6, 2026	October 9, 2026
Data Performed	Date Submitted

List the steps that you perform to come up with the program.

Paste the source code and explain the functionalities and take a screenshot of the final output.

OUTPUT/COMPILATION PROCESS:

```
/workspaces/bananabread (main) $ nasm -f elf32 slambook.asm -o slambook.o
/workspaces/bananabread (main) $ ld -m elf_i386 slambook.o -o slambook
/workspaces/bananabread (main) $ ./slambook

  ======================== MY SLAMBOOK ========================
  Enter one answer for every question.

  Name: Charles
  Email: cmarcelo@fit.edu.ph
  Blog/Website: None
  First big achievement: With High Honors
  First risk you ever took: Investing Money
  First time completely happy: With Friends in junior high
  Favorite color(s): Blue
  Favorite perfume: None
  Favorite music: Pop/Upbeat
  Favorite singer/band: Ben & Ben
  Favorite song: blue by Yung Kai
  Favorite food: Kare-Kare
  Weekend activity: Gaming, Sleeping, Watching
  Hobbies: Gaming, Music, Badminton, Drawing
  Favorite TV show: Young Sheldon
  Favorite movie: Not Sure
  Favorite book: Not Sure
  Favorite celebrity: Tom Holland
  Role model: None
  Ambition: Be financially free
  Motto: Your potential is your growth
```



SOURCE CODE:

section .data

intro db 10, '========== MY SLAMBOOK ==========', 10 db 'Enter one answer for every question.', 10, 10, 0

; Input prompts p_name db 'Name: ', 0 p_email db 'Email: ', 0 p_blog db 'Blog/Website: ', 0 p_achievement db 'First big achievement: ', 0 p_risk db 'First risk you ever took: ', 0 p_happy db 'First time completely happy: ', 0 p_colors db 'Favorite color(s): ', 0

p_perfume db 'Favorite perfume: ', 0 p_music db 'Favorite music: ', 0 p_singer db 'Favorite singer/band: ', 0 p_song db 'Favorite song: ', 0 p_food db 'Favorite food: ', 0 p_weekend db 'Weekend activity: ', 0 p_hobbies db 'Hobbies: ', 0 p_tv db 'Favorite TV show: ', 0 p_movie db 'Favorite movie: ', 0 p_book db 'Favorite book: ', 0 p_celeb db 'Favorite celebrity: ', 0 p_role db 'Role model: ', 0 p_ambition db 'Ambition: ', 0 p_motto db 'Motto: ', 0

; Output layout top db 10, '+----------+', 10, 0 title db '| MY SLAMBOOK |', 10, 0 divider db '|----------|', 10, 0 about db '| [ABOUT ME] |', 10, 0 faves db '| [MY FAVES] |', 10, 0 hobbies db '| [HOBBIES] |', 10, 0 ambition db '| [AMBITION] |', 10, 0 motto db '| [MOTTO] |', 10, 0

; Output labels l_name db '| Name : ', 0 l_email db '| Email : ', 0 l_blog db '| Blog/Website : ', 0 l_achievement db '| First big achievement: ', 0 l_risk db '| First risk taken : ', 0 l_happy db '| First completely happy: ', 0 l_colors db '| Color(s) : ', 0 l_perfume db '| Perfume : ', 0 l_music db '| Music : ', 0 l_singer db '| Singer/Band : ', 0 l_song db '| Song : ', 0 l_food db '| Food : ', 0 l_weekend db '| Weekend activity : ', 0 l_hobbies db '| Hobbies : ', 0 l_tv db '| TV Show : ', 0 l_movie db '| Movie : ', 0 l_book db '| Book : ', 0 l_celeb db '| Celeb : ', 0 l_role db '| Role model : ', 0 l_ambition db '| Ambition : ', 0 l_motto db '| ', 0 right_side db ' |', 10, 0 line_end db ' |', 10, 0 space_char db ' '

LINE_WIDTH equ 80 VALUE_WIDTH equ 35

art db 10 db ' / ', 92, **'', 10 db ' (o o', 92, '** _', 10 db ' / O', 10 db ' / (*/', 10 db ' //* U', 10,0

bottom db '+----------+', 10, 0

section .bss

; Each answer can contain up to 63 characters. b_name resb 64 b_email resb 64 b_blog resb 64 b_achievement resb 64 b_risk resb 64 b_happy resb 64 b_colors resb 64 b_perfume resb 64 b_music resb 64 b_singer resb 64 b_song resb 64 b_food resb 64 b_weekend resb 64 b_hobbies resb 64 b_tv resb 64 b_movie resb 64 b_book resb 64 b_celeb resb 64 b_role resb 64 b_ambition resb 64 b_motto resb 64

section .text global _start

; Print a zero-terminated string. ; Input: string address on the stack. print: push ebp mov ebp, esp push ebx push esi push edi

```
mov esi, [ebp+8]
mov edi, esi
```

.find_end: cmp byte [edi], 0 je .write inc edi jmp .find_end

.write: sub edi, esi mov eax, 4 ; sys_write mov ebx, 1 ; stdout mov ecx, esi mov edx, edi int 0x80

```
pop edi
pop esi
pop ebx
pop ebp
ret 4
```

; Print exactly the requested number of bytes. ; Input: string address and byte count on the stack. print_n: push ebp mov ebp, esp push ebx push esi

```
mov esi, [ebp+8]
mov edx, [ebp+12]
mov eax, 4          ; sys_write
mov ebx, 1          ; stdout
mov ecx, esi
int 0x80

pop esi
pop ebx
pop ebp
ret 8
```

; Return the string length, limited by the maximum on the stack. strnlen: push ebp mov ebp, esp push esi push ecx

```
mov esi, [ebp+8]
mov ecx, [ebp+12]
xor eax, eax
```

.count: cmp eax, ecx jae .done cmp byte [esi+eax], 0 je .done inc eax jmp .count .done: pop ecx pop esi pop ebp ret 8

; Print spaces to fill the rest of a fixed-width line. print_spaces: push ebp mov ebp, esp push ebx push esi

```
mov esi, [ebp+8]
```

.space_loop: cmp esi, 0 jle .done mov eax, 4 mov ebx, 1 mov ecx, space_char mov edx, 1 int 0x80 dec esi jmp .space_loop .done: pop esi pop ebx pop ebp ret 4

; Print one field without moving the right border. ; Input: label address, value address, maximum displayed value length. print_field: push ebp mov ebp, esp push ebx push esi push edi

```
; Get the label length.
mov eax, [ebp+8]
mov edi, eax
```

.label_length: cmp byte [edi], 0 je .label_ready inc edi jmp .label_length .label_ready: sub edi, eax mov ebx, edi

```
push dword [ebp+8]
call print

; Get the displayed value length, limited by VALUE_WIDTH.
push dword [ebp+16]
push dword [ebp+12]
call strnlen
mov esi, eax

push esi
push dword [ebp+12]
call print_n

; 80 columns total, including the final space and right border.
mov ecx, LINE_WIDTH - 2
sub ecx, ebx
sub ecx, esi
cmp ecx, 0
jle .no_padding
push ecx
call print_spaces
```

.no_padding: push dword line_end call print

```
pop edi
pop esi
pop ebx
pop ebp
```

```
ret 12
```

; Read one line into the buffer on the stack. ; This follows the reference style: one sys_read, then remove LF/CR. read_line: push ebp mov ebp, esp push esi

```
mov edi, [ebp+8]
mov eax, 3          ; sys_read
xor ebx, ebx        ; stdin
mov ecx, edi
mov edx, 63         ; maximum input length
int 0x80

test eax, eax
jz .empty

mov ecx, eax
xor esi, esi
```

.find_end: cmp byte [edi+esi], 10 je .terminate cmp byte [edi+esi], 13 je .terminate

```
inc esi
loop .find_end

mov byte [edi+esi], 0
jmp .done
```

.terminate: mov byte [edi+esi], 0 jmp .done

.empty: mov byte [edi], 0

.done: pop esi pop ebp ret 4

_start: push dword intro call print

```
push dword p_name
call print
push dword b_name
call read_line
push dword p_email
call print
push dword b_email
```

```
call read_line
push dword p_blog
call print
push dword b_blog
call read_line
push dword p_achievement
call print
push dword b_achievement
call read_line
push dword p_risk
call print
push dword b_risk
call read_line
push dword p_happy
call print
push dword b_happy
call read_line
push dword p_colors
call print
push dword b_colors
call read_line
push dword p_perfume
call print
push dword b_perfume
call read_line
push dword p_music
call print
push dword b_music
call read_line
push dword p_singer
call print
push dword b_singer
call read_line
push dword p_song
call print
push dword b_song
call read_line
push dword p_food
call print
push dword b_food
call read_line
push dword p_weekend
```

```
call print
push dword b_weekend
call read_line
push dword p_hobbies
call print
push dword b_hobbies
call read_line
push dword p_tv
call print
push dword b_tv
call read_line
push dword p_movie
call print
push dword b_movie
call read_line
push dword p_book
call print
push dword b_book
call read_line
push dword p_celeb
call print
push dword b_celeb
call read_line
push dword p_role
call print
push dword b_role
call read_line
push dword p_ambition
call print
push dword b_ambition
call read_line
push dword p_motto
call print
push dword b_motto
call read_line

; Display the completed slambook.
push dword top
call print
push dword title
call print
push dword divider
```

```
call print

push dword about
call print
push dword VALUE_WIDTH
push dword b_name
push dword l_name
call print_field
push dword VALUE_WIDTH
push dword b_email
push dword l_email
call print_field
push dword VALUE_WIDTH
push dword b_blog
push dword l_blog
call print_field
push dword VALUE_WIDTH
push dword b_achievement
push dword l_achievement
call print_field
push dword VALUE_WIDTH
push dword b_risk
push dword l_risk
call print_field
push dword VALUE_WIDTH
push dword b_happy
push dword l_happy
call print_field
push dword divider
call print

push dword faves
call print
push dword VALUE_WIDTH
push dword b_colors
push dword l_colors
call print_field
push dword VALUE_WIDTH
push dword b_perfume
push dword l_perfume
call print_field
push dword VALUE_WIDTH
```

```
push dword b_music
push dword l_music
call print_field
push dword VALUE_WIDTH
push dword b_singer
push dword l_singer
call print_field
push dword VALUE_WIDTH
push dword b_song
push dword l_song
call print_field
push dword VALUE_WIDTH
push dword b_food
push dword l_food
call print_field
push dword divider
call print

push dword hobbies
call print
push dword VALUE_WIDTH
push dword b_weekend
push dword l_weekend
call print_field
push dword VALUE_WIDTH
push dword b_hobbies
push dword l_hobbies
call print_field
push dword VALUE_WIDTH
push dword b_tv
push dword l_tv
call print_field
push dword VALUE_WIDTH
push dword b_movie
push dword l_movie
call print_field
push dword VALUE_WIDTH
push dword b_book
push dword l_book
call print_field
push dword VALUE_WIDTH
push dword b_celeb
```

```
push dword l_celeb
call print_field
push dword VALUE_WIDTH
push dword b_role
push dword l_role
call print_field
push dword divider
call print

push dword ambition
call print
push dword VALUE_WIDTH
push dword b_ambition
push dword l_ambition
call print_field
push dword motto
call print
push dword VALUE_WIDTH
push dword b_motto
push dword l_motto
call print_field
push dword divider
call print

push dword art
call print
push dword bottom
call print

mov eax, 1          ; sys_exit
xor ebx, ebx
int 0x80
```